

```

// =====
// ESP32-CAM COLOR TRACKING ROBOT
// Blue Triangle + Red Dot Detection System
// =====

// =====
// 1. LIBRARIES
// =====

#include "esp_camera.h"
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h";

// =====
// 2. CAMERA PIN CONFIGURATION
// AI Thinker ESP32-CAM
// =====

#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM   -1
#define XCLK_GPIO_NUM     0

#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM       5

#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23

```

```
#define PCLK_GPIO_NUM      22

// =====
// 3. FRAME SETTINGS
// =====

#define FRAME_W 160
#define FRAME_H 120

// =====
// 4. DIRECTION ZONE SETTINGS
// =====

#define CENTER_MIN  50
#define CENTER_MAX 110

// =====
// 5. OBJECT FILTER SETTINGS
// =====

#define MIN_BLOB  80
#define MAX_BLOB 8000

#define RED_TH 30

// =====
// 6. SERIAL SETTINGS
// =====

#define TXD2 14

// =====
// 7. GLOBAL VARIABLES
// =====
```

```

char lastSent = 'X';

// =====
// 8. COLOR DETECTION FUNCTIONS
// =====

// -----
// FUNCTION: isBlue()
// Checks whether a pixel is blue
// -----
bool isBlue(uint8_t r, uint8_t g, uint8_t b)
{
    return (b > r + 30 &&
            b > g + 20 &&
            b > 80);
}

// -----
// FUNCTION: isRed()
// Checks whether a pixel is red
// -----
bool isRed(uint8_t r, uint8_t g, uint8_t b)
{
    return (r > 160 &&
            r > g + 60 &&
            r > b + 60);
}

// =====
// 9. RGB EXTRACTION FUNCTION
// RGB565 → RGB888 Conversion
// =====

void extractRGB(uint16_t p,
                uint8_t &r,
                uint8_t &g,

```

```

        uint8_t &b)
{
    r = ((p >> 11) & 0x1F) << 3;
    g = ((p >> 5) & 0x3F) << 2;
    b = (p & 0x1F) << 3;
}

// =====
// 10. RED DOT VERIFICATION
// =====

// -----
// FUNCTION: checkRedDot()
// Checks whether a red dot exists near blue center
// -----
bool checkRedDot(camera_fb_t *fb, int cx, int cy)
{
    int redCount = 0;

    // Search box around center
    int xMin = max(0, cx - 20);
    int xMax = min(FRAME_W, cx + 20);

    int yMin = max(0, cy - 20);
    int yMax = min(FRAME_H, cy + 20);

    // Scan full frame
    for (int i = 0; i < fb->len; i += 2)
    {
        int idx = i / 2;

        int x = idx % FRAME_W;
        int y = idx / FRAME_W;

        // Skip pixels outside search area
        if (x < xMin || x > xMax ||

```

```

        y < yMin || y > yMax)
    {
        continue;
    }

    // Read pixel
    uint16_t p =
        (fb->buf[i] << 8) | fb->buf[i + 1];

    uint8_t r, g, b;

    extractRGB(p, r, g, b);

    if (isRed(r, g, b))
    {
        redCount++;
    }
}

return (redCount > RED_TH);
}

// =====
// 11. TRIANGLE SHAPE FILTER
// =====

// -----
// FUNCTION: isTriangleLike()
// Detects triangle-like blue object
// -----
bool isTriangleBetter(camera_fb_t *fb)
{
    int widthPerRow[FRAME_H] = {0};

    // scan image

```

```

for (int i = 0; i < fb->len; i += 2)
{
    int idx = i / 2;

    int x = idx % FRAME_W;
    int y = idx / FRAME_W;

    uint16_t p = (fb->buf[i] << 8) | fb->buf[i + 1];

    uint8_t r, g, b;
    extractRGB(p, r, g, b);

    if (isBlue(r, g, b))
    {
        widthPerRow[y]++;    // count pixels per row
    }
}

int prev = 0;
int increasing = 0;

// check row pattern
for (int y = 0; y < FRAME_H; y++)
{
    if (widthPerRow[y] > prev)
    {
        increasing++;
    }
    prev = widthPerRow[y];
}

// triangle-like condition
if (increasing > FRAME_H * 0.6)
    return true;

return false;
}

// =====
// 12. CAMERA SETUP

```

```

// =====

void setup()
{
    // Disable brownout reset
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    // Serial monitor
    Serial.begin(115200);

    // Communication with Arduino
    Serial2.begin(9600, SERIAL_8N1, -1, TXD2);

    // -----
    // Camera configuration
    // -----

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer   = LEDC_TIMER_0;

    // Camera data pins
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;

    // Control pins
    config.pin_xclk = XCLK_GPIO_NUM;

```

```

config.pin_pclk      = PCLK_GPIO_NUM;
config.pin_vsync     = VSYNC_GPIO_NUM;
config.pin_href      = HREF_GPIO_NUM;

config.pin_sscb_sda   = SIOD_GPIO_NUM;
config.pin_sscb_scl   = SIOC_GPIO_NUM;

config.pin_pwdn       = PWDN_GPIO_NUM;
config.pin_reset      = RESET_GPIO_NUM;

// Camera settings
config.xclk_freq_hz = 100000000;

config.pixel_format   = PIXFORMAT_RGB565;

config.frame_size     = FRAMESIZE_QQVGA;

config.fb_count       = 1;

// Initialize camera
if (esp_camera_init(&config) != ESP_OK)
{
    Serial.println("CAMERA FAILED");

    while (1);
}

Serial.println("CAMERA READY");
}

// =====
// 13. MAIN LOOP
// =====

void loop()
{
    // -----

```



```

// Capture frame
// -----

camera_fb_t *fb = esp_camera_fb_get();

if (!fb)
{
    return;
}

// =====
// BLUE OBJECT DETECTION
// =====

int blueCount = 0;

long sumX = 0;
long sumY = 0;

// Object boundary
int minX = FRAME_W;
int maxX = 0;

// -----
// Scan full image
// -----

for (int i = 0; i < fb->len; i += 2)
{
    int idx = i / 2;

    int x = idx % FRAME_W;
    int y = idx / FRAME_W;

    // Read pixel
    uint16_t p =

```

```

        (fb->buf[i] << 8) |
        fb->buf[i + 1];

uint8_t r, g, b;

extractRGB(p, r, g, b);

// -----
// Blue pixel detection
// -----

if (isBlue(r, g, b))
{
    blueCount++;

    sumX += x;
    sumY += y;

    // Update object boundaries
    if (x < minX)
    {
        minX = x;
    }

    if (x > maxX)
    {
        maxX = x;
    }
}

}

// =====
// OBJECT CENTER CALCULATION
// =====

int cx = (blueCount > 0) ? sumX / blueCount : -1;

```

```

int cy = (blueCount > 0) ? sumY / blueCount : -1;

// =====
// RED DOT DETECTION
// =====

bool redOK = (cx > 0) ? checkRedDot(fb, cx, cy) : false;

// =====
// TRIANGLE SHAPE DETECTION
// =====

bool triangleLike =
    isTriangleLike(
        blueCount,
        minX,
        maxX
    );

// Release frame buffer
esp_camera_fb_return(fb);

// =====
// MOVEMENT DECISION
// =====

char dir = 'S';

// Valid target condition
if (blueCount > MIN_BLOB && redOK && triangleLike)
{
    // Object at left
    if (cx < CENTER_MIN)
    {

```

```

        dir = 'L';
    }

    // Object at right
    else if (cx > CENTER_MAX)
    {
        dir = 'R';
    }

    // Object at center
    else
    {
        dir = 'F';
    }
}

// =====
// SEND COMMAND TO ARDUINO
// =====

if (dir != lastSent)
{
    Serial2.write(dir);

    lastSent = dir;
}

// =====
// DEBUG OUTPUT
// =====

Serial.printf(
    "Blue:%d CX:%d Red:%d Triangle:%d DIR:%c\n",
    blueCount,
    cx,
    redOK,
    triangleLike,
    dir

```

```
);
```

```
// Small delay
```

```
delay(15);
```

```
}
```

```
=====
=====
// =====
// Arduino Motor Controller
// =====
```

```
//
=====
// Arduino Motor Controller
//
=====
```

```
#include <SoftwareSerial.h>
```

```
// Software serial for ESP32-CAM communication
SoftwareSerial camSerial(4, 2); // RX=4, TX=2
```

```
// ===== MOTOR PINS =====
```

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11
#define ENA 5
#define ENB 6
```

```
// ===== SPEED SETTINGS =====
```

```
#define MAIN_SPEED 200
#define TURN_FAST 200
#define TURN_SLOW 80 // Lower = sharper turn
```

```
// ===== COMMAND TIMEOUT =====
```

```
#define CMD_TIMEOUT_MS 500
```

```
// ===== GLOBAL VARIABLES =====
```

```
char cmd = 'S';
```

```
char lastCmd = 'X';
unsigned long lastCmdTime = 0;

// ===== MOTOR FUNCTIONS =====

// Move forward
void forward()
{
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

    analogWrite(ENA, MAIN_SPEED);
    analogWrite(ENB, MAIN_SPEED);
}

// Turn left while moving
void turnLeft()
{
    // Left motor faster, right motor slower
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

    analogWrite(ENA, TURN_FAST);
    analogWrite(ENB, TURN_SLOW);
}

// Turn right while moving
void turnRight()
{
    // Right motor faster, left motor slower
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

    analogWrite(ENA, TURN_SLOW);
    analogWrite(ENB, TURN_FAST);
}
```

```
// Stop all motors
void stopMotor()
{
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
}
```

```
// ===== SETUP =====
void setup()
{
    Serial.begin(115200);
    camSerial.begin(9600);

    // Set motor pins as output
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    pinMode(ENA, OUTPUT);
    pinMode(ENB, OUTPUT);

    stopMotor();

    Serial.println("ROBOT READY");
}
```

```
// ===== MAIN LOOP =====
void loop()
{
    while (camSerial.available())
    {
        char incoming = (char)camSerial.read();
    }
}
```



```

// Ignore invalid commands
if (incoming != 'F' && incoming != 'L' &&
    incoming != 'R' && incoming != 'S')
    continue;

cmd = incoming;
lastCmdTime = millis();

// Ignore repeated command
if (cmd == lastCmd) continue;

Serial.print("CMD: ");
Serial.println(cmd);

// Execute command
switch (cmd)
{
    case 'F': forward(); break;
    case 'L': turnLeft(); break;
    case 'R': turnRight(); break;
    case 'S': stopMotor(); break;
}

lastCmd = cmd;
}

// Auto stop if connection lost
if (lastCmdTime > 0 && (millis() - lastCmdTime > CMD_TIMEOUT_MS))
{
    if (lastCmd != 'S')
    {
        Serial.println("TIMEOUT - stop");

        stopMotor();
        lastCmd = 'S';
    }
}

```